

AUTHORING TOOL DESIGN DOCUMENT OUTLINE

TITLE: *Sandglass*

PROJECT SUMMARY

Sandglass is an advanced authoring tool designed to simulate the behaviors and dynamics of sand with high fidelity, leveraging the Material Point Method (MPM) and the Drucker-Prager elastoplasticity model. The tool aims to provide artists and animators with an intuitive and efficient solution for creating realistic sand effects, including granular movement, terrain interaction, and external force influences.

Currently, creating realistic sand simulations in industry-standard software like Maya or Houdini is computationally expensive and technically complex. Most existing solutions require significant computational resources, making them impractical for real-time or interactive applications. Sandglass addresses this gap by offering a streamlined and accessible solution that balances accuracy, efficiency, and usability, making high-quality sand simulations more feasible for production environments.

Sandglass is primarily designed for artists and animators working in visual effects and animation. If real-time performance can be achieved, game designers could also benefit. In the Sandglass, users can initialize sand distributions (mass, positions, and velocity for particles), apply forces (e.g., gravity, wind), and generate sand motion over multiple frames.

The tool is built using MPM and the Drucker-Prager model for sand physics, integrated within Maya or Houdini. MPM is a hybrid grid-particle approach ideal for capturing granular flow and deformation, while Drucker-Prager is a well-designed robust elastoplasticity constitutive model that accurately models sand cohesion and movement. The effect of combination of these two techniques has been discussed in the paper *Drucker-Prager Elastoplasticity for Sand Animation* - Klar et al. 2016 .

By addressing the computational challenges of sand simulation and making it accessible within Maya or Houdini, Sandglass has the potential to become an

essential tool for digital artists aiming to achieve realistic, high-quality sand effects.

1. AUTHORING TOOL DESIGN

1.1. Significance of Problem or Production/Development Need

Simulating sand in digital content creation is a challenging task due to the complex behavior of granular materials. Traditional simulation techniques, such as particle-based methods (e.g., SPH), often struggle to replicate realistic sand flow, cohesion, and plastic deformation. MPM, combined with the Drucker-Prager elastoplasticity model, provides an efficient and physically accurate solution for these challenges.

Sandglass aims to bridge the gap between computational efficiency and visual realism, providing artists and animators with a user-friendly tool that simplifies the sand simulation process while maintaining high-quality results. By integrating directly into Houdini, Sandglass allows for seamless incorporation into existing production pipelines, making it accessible for both technical and creative users.

1.2. Technology

Sandglass is based on the Siggraph 2016 paper *Drucker-Prager Elastoplasticity for Sand Animation* by Klar et al. This paper presents an advanced MPM framework for simulating granular materials with realistic elastoplastic properties.

In this paper, it provides a physically accurate method for simulating sand dynamics, including stacking, flow, and cohesion. The Drucker-Prager model can effectively capture the unique deformation behavior of sand under different conditions. MPM allows for efficient simulation of large-scale granular materials without the instability issues of traditional particle-based approaches. By combining the MPM and Drucker-Prager model, this method is well-suited for integration into existing animation pipelines.

Specifically, The Material Point Method (MPM) is a hybrid particle-grid numerical technique widely used in computer graphics and computational physics to simulate deformable materials such as snow, sand, fluids, and soft bodies. In MPM, as shown in fig 1, material is represented as particles carrying mass, velocity, and

deformation information, while computations are performed on an underlying grid to avoid particle clustering and improve numerical stability. Each time step involves transferring particle data to the grid, solving physical equations on the grid, and then updating the particles.

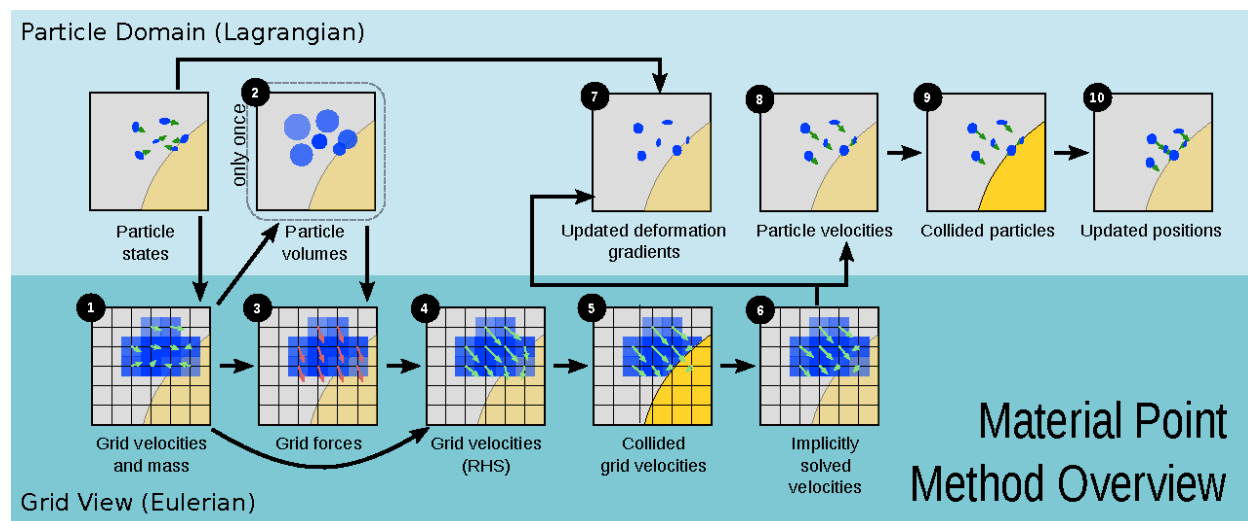


fig 1. The overview of MPM framework

The Drucker-Prager model is a mathematical model used in computational mechanics and computer graphics to simulate the behavior of granular and cohesive materials, such as sand, soil, and clay. The model defines a yield criterion in terms of the stress invariants, allowing it to describe how a material transitions from an elastic state to plastic deformation under different loading conditions. Unlike the models that use a conical failure surface with sharp corners, the Drucker-Prager model approximates this surface with a smooth circular cone (fig 2), making it numerically more stable and suitable for computational simulations.

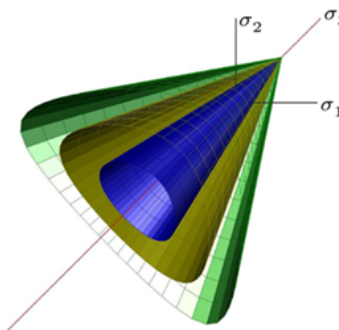


fig 2. Diagram for the "circular cone"

In the paper *Drucker-Prager Elastoplasticity for Sand Animation*, researchers have verified that the combination of MPM framework and Drucker-Prager constitutive model can simulate behaviors of sand very well. Some good effects are shown in fig 3, 4.



fig 3. Sand falls through the narrow neck of an hourglass, accumulating at the bottom.



fig 4. A stick is dragged through a bed of sand, tracing out a butterfly shape in the sand.

1.3. Design Goals

Sandglass is designed to cater to the needs of visual effects artists and animators by simplifying the process of sand simulation while maintaining a high level of physical realism. We hope the tool will provide an intuitive interface for configuring sand parameters and implement an efficient simulation engine based on MPM and the Drucker-Prager model. It should also support external forces such as gravity, wind, and user-defined interactions. After simulation, the tool is expected to generate simulation data that can be exported for high-quality rendering.

Ideally, it can also allow real-time visualization and interactive adjustments to simulation parameters. If real-time performance can be optimized, game designers could also leverage Sandglass to achieve more dynamic and responsive sand effects in interactive environments.

1.3.1 Target Audience.

The primary users of Sandglass include visual effects artists, animators, and game developers. They can use Sandglass for creating natural-looking sand effects in films, advertisements, and digital content. If performance optimizations are successful, it could be used for real-time sand simulations in interactive media.

1.3.2 User Goals and Objectives

Users should be able to define initial sand distributions (including particle mass, position, and velocity) and configure simulation parameters such as material properties and external forces. After these operations, users should expect to preview and interact with the simulation in real-time. When they are satisfied with simulation effects, they can choose to export high-fidelity simulation data for further processing and rendering.

1.3.3 Tool Features and Functionality

Firstly, the tool can provide a GUI for users to set properties for sand particles. It should also provide a way for users to import these data from a prepared file in particular format. Additionally, Sandglass should also allow users to adjust friction, cohesion, plasticity, and other material properties.

After getting the parameters needed in simulation, Sandglass could execute the simulation based on the method and model given in paper *Drucker-Prager Elastoplasticity for Sand Animation*.

When simulation is done, the tool can show a live preview of the simulation and export the data for rendering.

1.3.4 Tool Input and Output

Input: User-defined sand distributions, initial conditions, material properties, and environmental settings.

Output: Simulated sand particle movement over multiple frames, exportable as particle data for rendering.

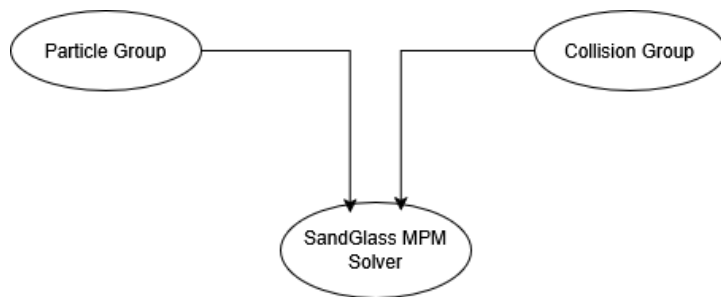
1.4. User Interface

In this section you need to describe what the user interface looks like and how will it be used. You also need to provide a mockup of the interface

1.4.1 GUI Components and Layout

- *What are the main GUI features in terms of windows, property panels and menu items? Show the physical layout of the GUI and describe the functionality of each of its components*

Houdini Nodes



MPM Node UI	
Material Properties	Simulation Properties
Young's Modulus	Grid Height
1.4 × 10^5	5
Poisson Ratio	Gravity
0.2	9.8
Friction Angle	Integration
30	Explicit
	Time Step
	0.01

1.4.2 User Tasks

Tasks

- Users will be able to connect a particle groups to the MPM simulation
- Users will be able to connect predefined force equations to the MPM
- Users will
- Users will be able to edit the parameters of the simulation
- Users will be able to step through the simulation

Preexisting Knowledge

- Users will need knowledge of Houdini, how to work with nodes, create particles, define a collision group, and step through the timeline.
- If users want to attach custom , they will need to be familiar with the layout of MPM, so that they can properly retrieve the correct information from the nodes within the MPM solver to allow them to calculate the force

1.4.3 Work Flow

- *What does the tool require as input and what does it produce as output?*

Basic Steps:

1. Create new Houdini Project.
2. Create new Geometry node.
3. Create new Sandglass MPM Node
4. Define material and simulation properties
5. Connect POP node to MPM Node (Initial Conditions + Sand Distributions)
6. If they want additional forces, go within the MPM Solver and define custom forces, connecting to force merge wrangle node (Initial Conditions + Environmental Settings)
7. Connect Collision Objects to MPM Node (Environmental Settings)
8. Run simulation by stepping through timeline

2. AUTHORIZING TOOL DEVELOPMENT

2.1. Technical Approach

2.1.1 Algorithm Details

1. APIC transfer, transferring the speed and momentum of the particles onto a 3D grid, as well as transferring density and volumes from the grid back to the particles
2. Internal velocity calculations, taking in the material properties and outputting a force at each grid point.
3. Velocity calculation, both explicit and implicit. Explicit straightforward, velocity + acceleration calculated from force, implicit will need to be solved using GMRES
4. Deformation update. Using Drucker Prager plasticity model to calculate both the elastic and plastic deformation.
5. Collisions. Calculate collisions of the grid nodes with objects in the world, with different types of collisions based off the object.
6. Particle update, update a large set of particles with their velocities.

2.1.2 Maya or Houdini Interface and Integration

We plan to implement the algorithms in Houdini, as Houdini is generally seen as a better environment for implementing simulations. The basic structure goes as follows

1. Generate our collection of particles. This may be done a variety of ways using built in Houdini nodes, such as a volume to particle node.
2. Transfer mass and momentum from the particles to a grid, in this case represented by a series of Houdini Fields, which are a collection of vectors at specific points. As this is a simple operation applied on all particles for each node, we can write a VEX program within a wrangle node for this.
3. Transfer density back from the Field to our particles, similar as above, but with a geometry wrangle node. As we only need the initial volumes, this step only needs to happen once during the start of the simulation
4. Calculate internal forces. Taking in the deformation Fields, initial volumes, and material properties, we should be able to calculate the force by directly plugging them into a formula and using the built in SVD nodes within Houdini.
5. Sum up forces. Wrangle together the external forces and the internal forces for each material point in the field.
6. Explicit velocity. This can be done in a single wrangle node, using the summed up forces and velocities at each material point in the field.
7. Collision, use Houdini's collision objects to define the areas of collision. Figure out what parts of the field are colliding, then calculate the collision in a custom node written in the HDK.
8. Implicit velocity. Same inputs as above, however will require some math libraries to solves for GMRES to solve the system, and as such will be done, and as such will be done in a custom node written in the HDK. There will be a boolean option for switching between explicit and implicit.
9. Calculate Friction: Done after the implicit velocity as the collision itself can be integrated into it, applies the friction to the new velocity.
10. Calculate Deformation field. Since the Drucker Prager MPM requires the calculating of the principle stresses, this will be done in a node written in the HDK.
11. Transferring grid velocities back to the particles, similar as above can wrangle it up and apply the weighting operation with a VEX program.
12. Update particles. Velocities from solver are plugged into our particles group.

2.1.3 Software Design and Development

1. APICParticleToGrid wrangle node: Transfers the mass and momentum from the particles to a Houdini field
2. APICGridDensityToParticle wrangle node: transfers the initial density from the field back to the particles, and calculates the initial volume for each of the particles
3. InternalElasticForce node: Uses the initial volume, elastic deformation fields, and material coefficients to calculate the internal elastic force at each of the points of the field.
4. Explicit Velocity wrangle node: Takes in the force, mass, and velocity of the field + the timestep and calculates the explicit velocity.
5. GridCollision Node: Takes in the explicit velocity and collision objects, and checks each particle for collision, calculates a new velocity based off the collisions
6. Implicit Velocity Node: Takes in the explicit velocity and forces and uses it to implicitly calculate the velocity, using GMRES
7. Friction Node: Takes the velocity, and applies friction to it.
8. Deformation Field Node: Take the previous deformations fields and the velocity and calculate new deformation fields.
9. APICGridVelocityToParticle wrangle node: transfers velocity from the field back to the particles.

Implementation Note: Since the houdini fields can only hold a singular vector, the field is actually a collection of fields spanning the same range, mainly a velocity field and a mass field.

Third Party Software:

ALGLIB: Third party library for calculating GMRES

2.2. Target Platforms

2.2.1 Hardware

Minimal hardware to run Houdini 20.5, see details here
<https://www.sidefx.com/Support/system-requirements/>

2.2.2 Software

Windows 10/11 and Houdini 20.5.

2.3. Software Versions

2.3.1 Alpha Version Features (first prototype)

- In Alpha Version, the tool has to be able to receive the parameters correctly from GUI and user input. Based on that, the plugin should be able to run a simplified explicit MPM framework, including Particle to

Grid step, Grid to Particle step, force/velocity calculations, and deformation update. To test this version of Sandglass, we plan to implement a Corotated elastic model, which can simulate simple behaviours of elastic bodies.

- At this stage, the tool is expected to print all user inputs to verify they are parsed correctly. Then, based on the position of particles, the tool should simulate them as an elastic body to test the implementation of the MPM framework.

2.3.2 Beta Version Features

- The main task in Beta Version should be applying the Drucker-Prager model on the material. To be specific, during the “Deformation Update” step in the MPM framework, we should replace the old elastic model with Drucker-Prager model to simulate the behavior of sand. Furthermore, we should also make the tool be able to export the data in this step.
- To test the Beta Version, we can try to reproduce some results in the paper *Drucker-Prager Elastoplasticity for Sand* using the workflow of the tool and export the data which can be used for rendering.

2.3.3 Description of any demos or tutorials

- Actually, the usage of Sandglass and the meanings of parameters are intuitive. There is also no need for post processing because the sand is granular material. However, to make sure it is a user-friendly tool, we will provide detailed documentation and demo videos for using our tool.
- To ensure that users can effectively utilize Sandglass, we will provide a comprehensive set of demos and tutorials that cover various aspects of the tool's functionality. These materials will illustrate the workflow of the tool and the meaning of parameters. We hope users can understand how to adjust the parameters to get the desired results.

3. WORK PLAN

3.1. Tasks and Subtasks

- List and number all the tasks and subtasks which are necessary to develop your authoring tool. **Each main task should be comprised of at least 3 subtasks.**
- Provide **separate descriptions of each task and subtask, which members of the group are assigned to it, and the expected task duration.** Be as detailed as you can.
- Use the following format for each of your Work Plan tasks.

Task 1 – Framework Setup

1. **Setup Houdini:** Setup basic particle system, get familiar with inputs and outputs
2. **Test built in MPM:** Test existing MPM, see if there are any built in utilities we can extract from it.

Task 2 – Basic MPM

1. **APIC Transfer:** Transform particle states to field
2. **Explicit Velocity (Only Gravity):** Calculate grid velocity and transfer back to particles, using gravity as the only force.
3. **Internal forces:** Use simple corotated energy density model to calculate the internal forces. Introduce deformation gradient, only elastic deformation.
4. **Elastic Plastic separation:** Separate deformation into elastic and plastic parts, use principle stretches for calculation (will be shifted later to principle stresses)
5. **Collision:** Develop a model for handling simple collisions on the grid level. Incorporate with Houdini's collision objects.

Task 3 – Drucker Prager constitutive model

1. **Swap energy model:** Switch energy density function to the one used in the Drucker Prager Elastoplasticity for Sand Animation
2. **Use Drucker Prager's method for plasticity:** Instead of having cutoffs in the stretching and compressing, project to yield surface.
3. **Separate Friction:** Separate friction from the collision model. Done in anticipation of implicit integration.
4. **Incorporate other types of collision:** Develop models for sticky and deforming collisions. Create collision groups for the objects that have different kinds of collisions.
5. **Develop GUI:** Create a GUI that localizes all the system parameters into one place.

Task 4 – Implicit Integration + Overhead

- 1. Implement Implicit Integration Method:** Arrange into linear system and solve using GMRES
- 2. Create Import/Export system:** Allow importing/exporting of particle data and material parameters.
- 3. Create Demos:** Develop demos showcasing utility and accuracy of system. See if we can find the demos used by the paper itself.

3.2. Milestones

3.2.1 Alpha Version

Tasks #1 and #2 complete

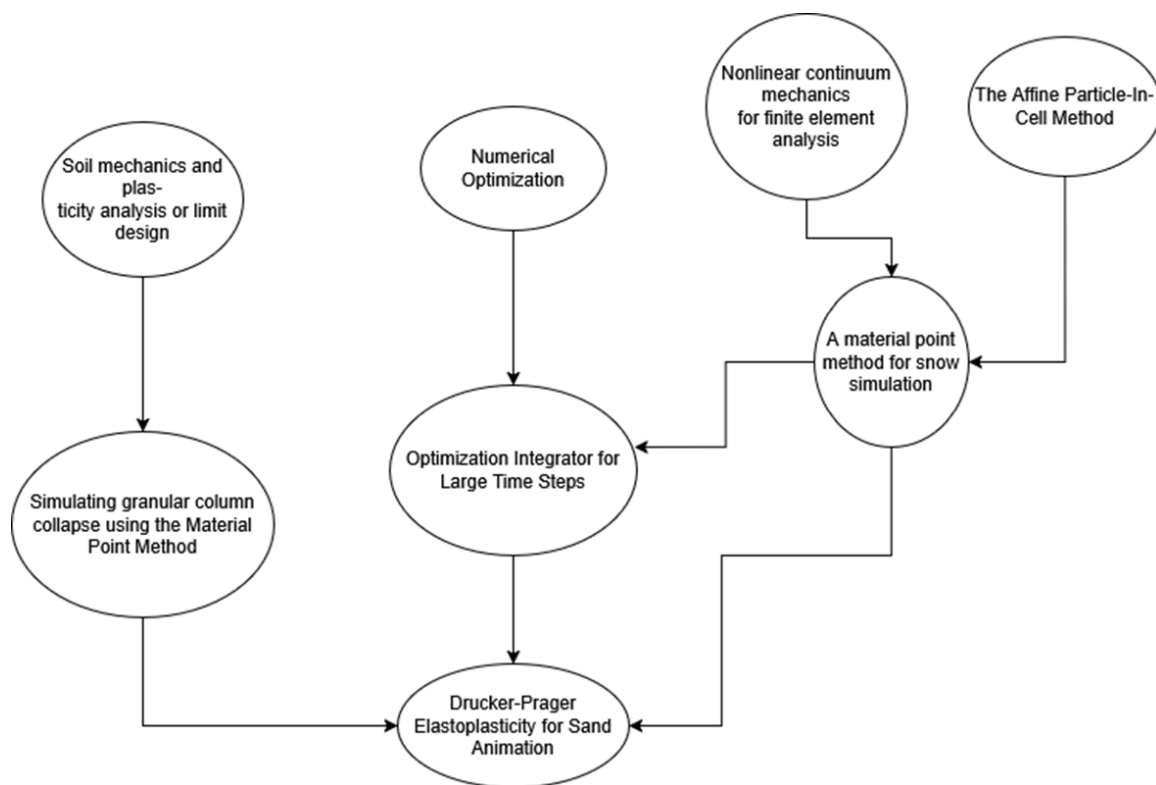
3.2.2 Beta Version

Tasks #3 complete, #4 optional except for the demos, which should be completed

3.3. Schedule

https://docs.google.com/spreadsheets/d/1nBDKEQkD4W476J_DpJzNKMjkt3D7xIAjdOK62g0C4x0/edit?usp=sharing

4. RELATED RESEARCH



[Nonlinear continuum mechanics for finite element analysis. Cambridge University Press, Bonet, J., and Wood, R. 2008]

This paper provides the basis for the Continuum Mechanics. This includes the Kirchhoff stress tensors, the equations for Hyperelasticity, and how to decompose a deformation into plastic elastic parts. So, essentially the base equations for relating energy density to stress to internal forces in a Eulerian view, and using it to calculate deformations (as seen in MPM), is given by this paper. The main paper also uses these same basic principles in their implementation of MPM.

[Soil mechanics and plasticity analysis or limit design, Drucker, D., and Prager, W.]

The Drucker Prager paper gives the constitutive model for modelling the stress-strain relationship in soil. While the original MPM paper chooses to ignore calculating the principal stresses in favor of calculating the principal stretches, the main paper chooses to calculate these principal stresses, using the same methodology Mast et al used, who derived their plastic constitutive model from this paper. An important thing this paper adds to the constitutive model is the friction between the sand particles.

[Numerical Optimization, Nocedal J. and Wright S.]

Provides methodology for solving problems as optimization problems. Provides the necessary background for Gast et al to solve their implicit integration schema. Basically a whole textbook on numerical optimization theory.

[Material point method for snow simulation, Stomakin et al]

This paper introduced the material point method into graphics, setting up a basis for MPM in a graphics setting which is used by the main paper. It defines a constitutive equation for an elastoplastic material with principal stretches and a hardening coefficient,

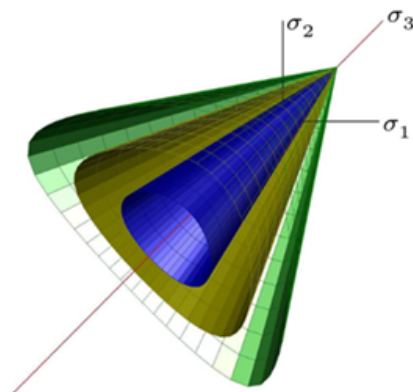
divides the deformations into elastic and plastic parts, and utilizes APIC for transferring mass and momentum from the particles to the grid and vice versa.

[The Affine Particle-In-Cell Method, Chenfanfu J. et. al]

This paper introduces a new form of transformation we can use in MPM. In fact, instead of transferring the velocity of particles directly, APIC adds an “affine” matrix for every particle. This matrix can store the information of the rotations of each particle, which can be used to make angular momentum conservative. In the “Particle to Cell” step we need to utilize this matrix to compute the cell velocities. In the “Cell to Particle” step, we should maintain the correct affine matrix for particles by updated cell velocities.

[Simulating granular column collapse using the Material Point Method, Cater M. Mast et. al]

This paper gives a constitutive model we can use in MPM to simulate phenomena of sand. The basic idea of this constitutive model is that in each dimension, sand would be elastic under a compression and plastic under a stretch. Furthermore, if it's compressed too much it would be plastic deformation, too. Thus, in a coordinate system consisting of singular values σ of the deformation gradient matrix, only a triangular area in the left-bottom part corresponds to elastic deformation. Similarly, in 3D case the shape of this area becomes a cone:



At the same time, this paper also describes how to calculate the hardening parameter, which is really useful in keeping the shape of the surface.

[Optimization Integrator for Large Time Steps, Theodore F. Gast et. al]

This paper shows how simple collisions can be incorporated directly into the solver through constrained minimization without sacrificing efficiency. In section 6, it discusses how to handle the collisions and external forces as a minimization task in the MPM framework.

There are three different types of collisions. The first one is collisions with analytic objects, in this case we can consider the collisions as a kind of constraints. The second case is objects are not analytic, in which we need to use a penalty formulation instead. At last, we need to handle the self-collisions in simulating soft bodies. In our project we aim to simulate sand so it is not necessary to handle self-collisions.

AUTHORING TOOL Final Report

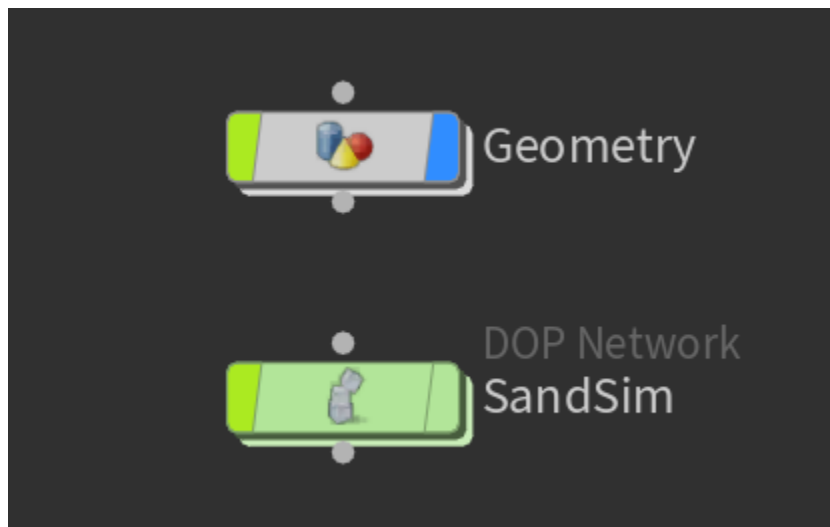
Qirui Fu, Pavel Peev

Project Name : Sandglass

1. Results

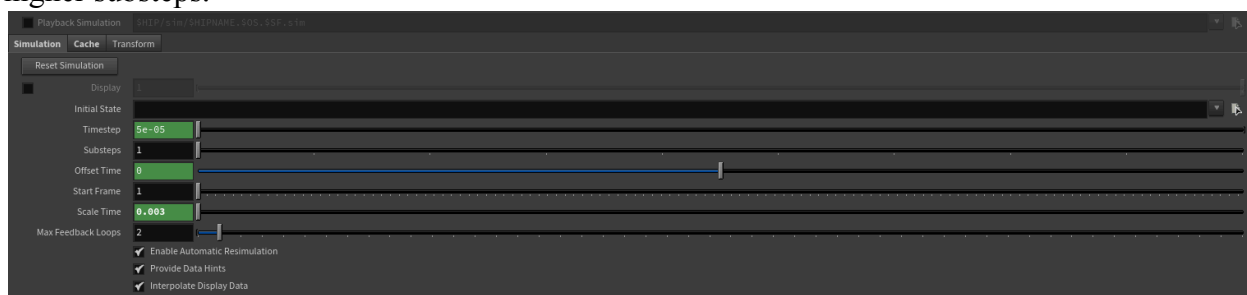
1.1 Documentation + Content Creation

We provide a .hipnc file containing our Houdini simulation. In the node network, we have a DOP network that handles the simulation, and a geometry node which contains the geometry that interacts with the simulation.dfsf

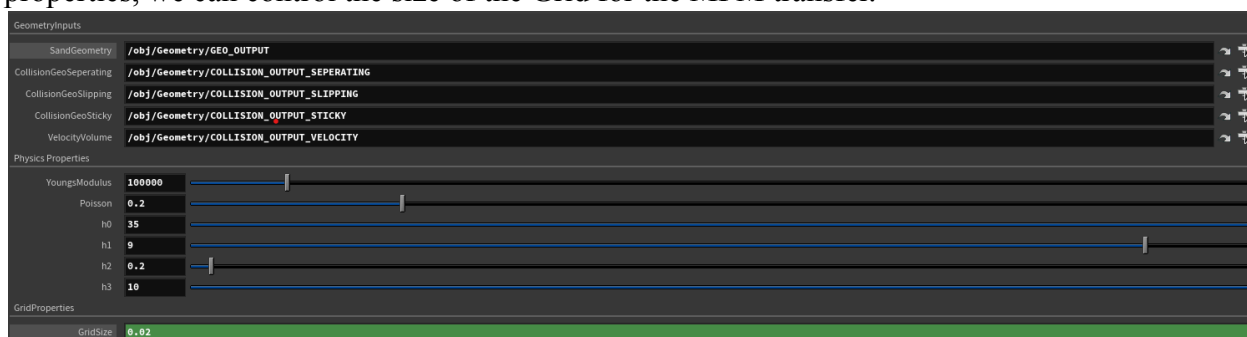


CIS6600: Advanced Topics in Computer Graphics and Animation

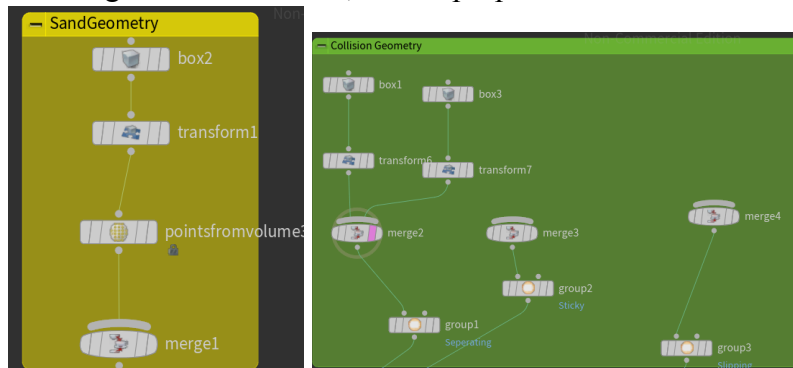
At the top section of the DOP network, we have the general DOP parameters controlling the time step. The time scale should be set to a low amount, generally under 0.005, and the substeps should be left at 1 due to a general bug that causes the volume to not be calculated correctly at higher substeps.



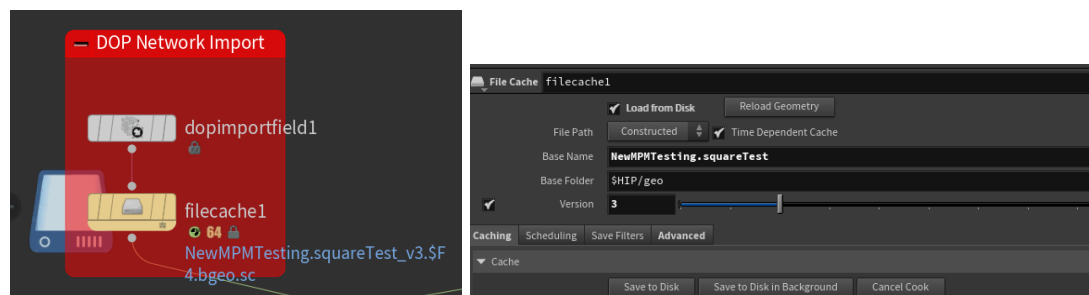
At the bottom section, we have our physics inputs. Under the geometry inputs section, we specify the paths to our sand geometry and our collision geometry. Under our physics properties, we have the properties used in the calculation of the physics of the sand. Finally, under grid properties, we can control the size of the Grid for the MPM transfer.



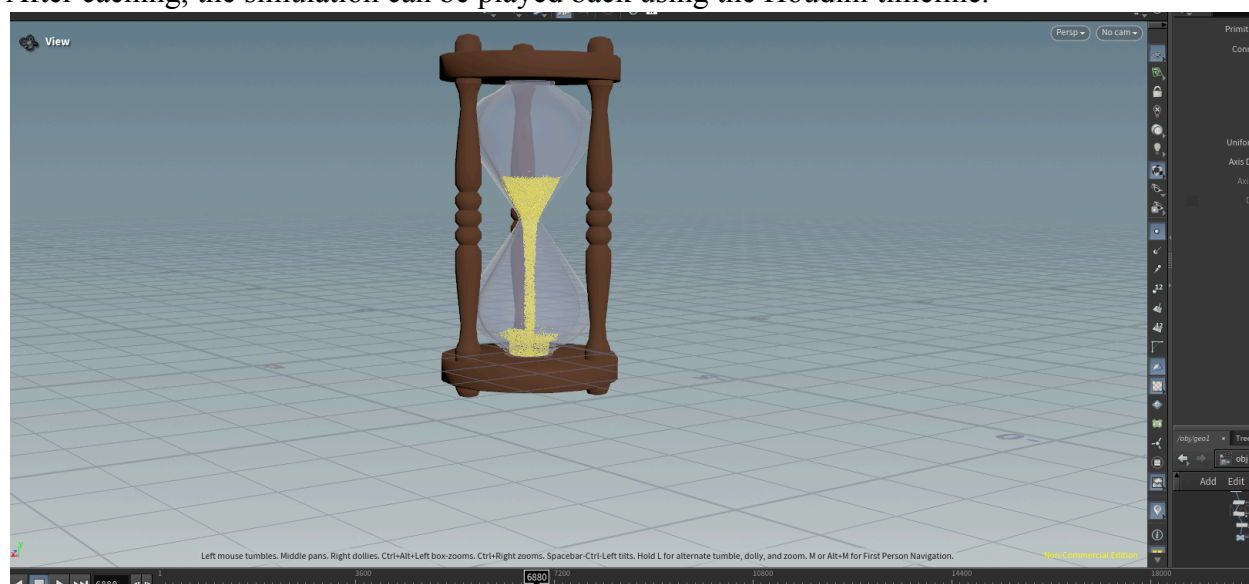
In the geometry node, we have 3 important areas. The first area is where the sand geometry is created. This can take any collection of particles, we recommend using the pointsfromvolume node to convert the geometry into particles. You can also look at the further nodes if you want to change any properties such as mass and velocity of the sand. The second area is where our collision geometry is processed. Here, we can plug in any geometry into the different areas corresponding to sticky, separating, and slipping collision. Further down the line, it gets turned into a signed distance field, whose properties can be edited if needed.



Finally, we have the area where the simulation can be cached to disk. By providing a file path and clicking save to disk, we're able to cache the entire simulation.



After caching, the simulation can be played back using the Houdini timeline.



2. Discussion

2.1 Accomplishments

In this project, we successfully implemented a Houdini plugin which can simulate the behaviors of sand. In our tool, users can just set the initial state of sands and the position of objects. After that, our tool can handle the collisions between sand and objects and get a realistic simulation result.

Overall, we accomplished most of the goals outlined in the design doc. Firstly, our tool allows users to set the initial outline geometry of sand, external forces like gravity, and parameters including friction angle to control the simulation. Then, using the MPM framework with the Drucker-Prager constitutive model, our tool is

able to simulate the movement of sand. Finally, users can cache the simulation results so they can use them to make animation or special effects as they want.

However, there are some features that we are not able to implement in this project. The most significant one is the implicit update scheme of MPM. Since implicit MPM is much more stable, we can obtain larger time steps with it and increase the efficiency of simulation. After learning a lot about implicit MPM, we have understood the workflow of it, but we found it is too complicated to implement it in just a few weeks. As a result, we decided to implement explicit MPM only.

2.2 Design Changes

There are not many changes to our tool compared with the Design Doc. For GUI, our final version is more complex than we thought, which means more parameters for users to tune. The workflow is also consistent with what we described in the Design Doc.

2.3 Total Man-Hours Worked

Qirui Fu : about 45 hours.

Pavel Peev : about 150 hours

2.4 Future Work

Although we have implemented the basic functions and our tool works well, there are still a lot of things we can do in the future.

First, implicit MPM is a meaningful feature we should try. As we said before, it's hard to implement it but we can increase the efficiency of our tool magnificently with it.

Additionally, we could also try to add more properties and materials in our tool. For example, we can try to simulate the elastic body and snow in our tool. And we can also try to allow users to simulate different types of sand using our tool, such as wet sand which has higher friction.

We could also add some presets for different types of sand, to make it easier

2.5 Third Party Software

Only made with Houdini nodes and Houdini's Vex language. Architecture was originally inspired by <https://nccastaff.bournemouth.ac.uk/jmacey/MastersProject/MSc20/03/thesis.pdf>, with a lot of improvements to it on our end.

3. Lessons Learned

In this project, we successfully implemented some features as planned and also met some challenges.

An important lesson is to first try implementing features in a simpler environment. Programming in Houdini is an annoying thing because it's hard to debug and determine which part doesn't work. For that reason, in this project we always implemented features in python at first, then ported them to Houdini. In this way, we can make sure we completely understand the algorithm and have a correct reference for Houdini.